

Regenbogen-Artefakte beim Isolieren texturierter Handscans

Fehlersuche, verworfene Ansätze und die tatsächliche Ursache

Simon Bichler

September 2026

1 Problembeschreibung:

Beim Isolieren eines einzelnen Fingers aus einem vollständigen Handscan traten in der 3D-Ansicht auffällige Artefakte auf, und zwar gezackte, mehrfarbige Linien, die sich wie aufgestickte Nähte über die Fingeroberfläche zogen. Sie folgten sichtbar den Dreieckskanten des Meshes und bestanden aus mehreren dünnen, unterschiedlich gefärbten Streifen nebeneinander.

Das Verhalten war reproduzierbar und trat ausschließlich bei den neueren Scans auf, die mit RealityCapture erzeugt wurden und eine einzige große Textur (8192×8192 Pixel) aus einer einzelnen PNG-Datei verwenden. Ältere Scans, deren Textur aus meist vier JPG-Dateien besteht, zeigten das Problem nicht. Im vollständigen, ungeschnittenen Handscan war ebenfalls nichts zu sehen. Die Artefakte entstanden erst beim Isolieren.

Nach dem Schneiden des Meshes zeigten rund 11 % der verbleibenden Dreiecke Texturinhalt, der nicht zu ihrer Position auf der Oberfläche gehört.

2 Probleme bei der Fehlerbehebung:

Das Artefakt hatte eine unglückliche Eigenschaft. Es sah nach einem Textur- oder Beleuchtungsproblem aus, war aber keines von beidem. Dadurch führten mehrere plausible Hypothesen nacheinander nicht zu einer Lösung. Bei einer ausführlichen Recherche im Internet fand ich keine Quelle, die diesen Fehler ebenfalls beschreibt.

3 Verworfene Lösungsansätze

Alle folgenden Ansätze wurden umgesetzt und am realen Datensatz geprüft. Keiner davon hat das Artefakt verändert.

Ansatz	Annahme dahinter	Messergebnis
UV-Werte als Vertexfarben speichern	Spürge entstehen beim Export	Eigenständiger Bug, aber andere Ursache
Vertexfarben statt Textur	Textur ganz umgehen	Auflösung bricht auf Vertexdichte ein
Verschmelzungstoleranz auf 0	Punkte werden zu großzügig verschmolzen	Wirkungslos: deckungsgleiche Punkte werden auch bei Toleranz 0 verschmolzen
Normalen nicht auto-orientieren	Umgeklappte Flächen scheinen durch	Keine Änderung
Gar keine Punktnormalen	Schattierungskanten	Keine Änderung
Ohne Beleuchtung rendern	Beleuchtungsartefakt	Keine Änderung; beweist: Texturinhalt
Bilineare Filterung + Mipmapping	Aliasing der Nearest-Abtastung	Keine Änderung
Textur auf 4096 px verkleinern	Treiber- oder Größengrenze	Keine Änderung
Rand der UV-Inseln um 8 bzw. 64 px verbreitern	Dreiecke tasten Atlas-Hintergrund ab	Keine Änderung
Ungenutzte Texel transparent setzen	Fremdinhalt aus dem Atlas fernhalten	Logisch ausgeschlossen (s. u.)
Anzeige aus dem Speicher statt aus der Datei	Verlust beim OBJ-Export	Identisch, also nicht der Export
Naht-Harmonisierung der Farben	Belichtungsunterschied zwischen Fotos	Mittlere Differenz ≈ 0 , kein systematischer Versatz
Neu texturieren im eigenen Programm	Textur selbst erzeugen	Unmöglich ohne Fotos und Kamerapositionen

Die Beleuchtung war unschuldig. Rendern ohne jede Beleuchtung zeigte die Streifen unverändert. Damit war klar, dass es sich um Texturinhalt handelt und nicht um Schattierung.

4 Korrekte Lösungsfindung mit experimentellem Ansatz:

Die offene Frage war, ob der Fehler in den Daten an den Eckpunkten der Dreiecke liegt oder erst beim Rendern zwischen ihnen entsteht. Um das zu trennen, wurde dasselbe geschnittene Mesh zweimal dargestellt mit identischen Vertexpositionen, Dreiecken, UV-Koordinaten und derselben Textur dargestellt. Der einzige Unterschied war, dass einmal die Textur klassisch über die UV-Koordinaten gelegt wurde, sodass das Dreiecksinnere die Textur abtastet. Danach wurde die Farbe vorab an jedem Vertex aus derselben Textur abgetastet und fest als Vertexfarbe zugewiesen, sodass zwischen den Eckpunkten nur noch übergeblendet wird.

Läge der Fehler an den Eckpunkten, müsste er in beiden Fällen auftreten. Tatsächlich zeigte nur das Texturrendering die Streifen. Mit den abgetasteten Vertexfarben verschwanden sie vollständig. Der Fehler entsteht also erst im Inneren der Dreiecke, wo in der Textur interpoliert wird, die Eckpunkte selbst tragen korrekte UV-Koordinaten.

Eine Messung bestätigte diesen Gedanken. Betrachtet man je Dreieck, wie weit die UV-Koordinaten seiner Ecken auseinanderliegen, also die größte UV-Kante, hatte im originalen Handscan, welcher aus 149 986 Dreiecken bestand, kein Einziges einen Wert über 0,028. Im isolierten Finger (13 450 Dreiecke) überschritten dagegen 1 555 Dreiecke diesen Wert, der größte

erreichte 1,043. Da UV-Koordinaten auf 0 bis 1 normiert sind, bedeutet ein solcher Wert, dass ein einzelnes Dreieck über die gesamte Texturbreite gespannt ist und sein Inneres alles Dazwischenliegende abtastet, wodurch die mehrfarbigen, Regenbogenartigen Streifen entstanden.

5 Die tatsächliche Ursache:

Ein Texturatlas ist in viele einzelne Inseln aufgeteilt. An deren Rändern muss das Mesh doppelte Vertices führen, und zwar zwei Punkte auf exakt derselben 3D-Position, aber mit zwei verschiedenen UV-Koordinaten, eine pro Inselseite. Im untersuchten Scan waren das 17 635 solcher Paare.

Beim Schneiden mit `clip_surface` erzeugt VTK neue Punkte auf der Schnittkante und verschmilzt dabei deckungsgleiche Punkte zu einem einzigen. Genau das trifft die Nahtvertices, wobei von den beiden UV-Koordinaten nur eine überlebt. Die Dreiecke der anderen Seite zeigen anschließend auf die falsche Insel.

Das Verschmelzen ist nicht abschaltbar, indem man die Toleranz auf null setzt, Punkte mit Abstand exakt null werden auch dann zusammengelegt. Der Fehler lässt sich nur vermeiden, indem beim Schneiden gar keine neuen Punkte entstehen.

6 Die korrekte Lösung:

Statt die Dreiecke an der Ellipsoidfläche zu zerschneiden, werden nur noch komplette Dreiecke behalten oder verworfen. Ohne neue Punkte gibt es nichts zu verschmelzen, und die UV-Zuordnung bleibt unangetastet.

```
1 def waehle_ganze_dreiecke(mesh, flaeche, invert=False):
2     # Behält ganze Dreiecke, statt sie an der Schnittflaeche zu zerschneiden.
3
4     mit_abstand = mesh.compute_implicit_distance(flaeche)
5     abstand = np.asarray(mit_abstand["implicit_distance"])
6     dreiecke = mit_abstand.faces.reshape(-1, 4)[: , 1:]
7
8     if invert:
9         behalten = (abstand[dreiecke] <= 0).all(axis=1)
10    else:
11        behalten = (abstand[dreiecke] >= 0).all(axis=1)
12
13    if not behalten.any():
14        return p_v.PolyData()
15
16    ausgewaehlt = mit_abstand.extract_cells(behalten).extract_surface()
17
18    # extract_surface() verliert die Kennzeichnung als aktive Texturkoordinaten
19    if "Texture Coordinates" in ausgewaehlt.point_data:
20        ausgewaehlt.active_texture_coordinates = np.asarray(
21            ausgewaehlt.point_data["Texture Coordinates"]
22        )
23
24    return ausgewaehlt
```

Listing 1: Auswahl ganzer Dreiecke statt echtem Schnitt

Verifikation

	bisher	neu
Defekte Dreiecke (Testschnitt)	447 von 2 427	0 von 2 243
Größte UV-Kante	1,004	0,006
UV-Koordinaten gegenüber Original	verändert	bitgenau identisch
Positionen gegenüber Original	interpoliert	bitgenau identisch

Auch im gerenderten Vergleich sind die Streifen restlos verschwunden.

Preis der Lösung

Der Rand des isolierten Fingers verläuft nun entlang der Dreieckskanten statt exakt auf der Ellipsoidfläche. Bei einem typischen Vertexabstand von etwa einem halben Millimeter fällt das optisch kaum auf, verschiebt die Volumenmessung aber geringfügig, allerdings systematisch gleichsinnig über alle Scans hinweg, sodass Verläufe untereinander vergleichbar bleiben. Außerdem werden dann nicht Volumina erfunden, sondern folgen exakt aus den Messungen mit den jeweiligen 3D-Kameras.

Wer den exakt glatten Schnitt braucht, kann das Mesh vor dem Schneiden nach UV-Inseln aufteilen und jede Insel einzeln schneiden. Dann liegen die doppelten Nahtvertices in getrennten Teilmeshes und können nicht mehr verschmolzen werden.